

6.4 List methods ^

Settings

Beta features

list method

A list method can perform a useful operation on a list such as add sorting, reversing, etc.

Table 6.4.1: Available list methods.

Adding elements

List method	Description	Code example
list.append(x)	Add an item to the end of list.	<pre>my_list = [5, 8] my_list.append(16)</pre>
list.extend([x])	Add all items in [x] to list.	<pre>my_list = [5, 8] my_list.extend([4, 12])</pre>
list.insert(i, x)	Insert x into list <i>before</i> position i.	<pre>my_list = [5, 8] my_list.insert(1, 1.7)</pre>

Removing elements

List method	Description	Code example
list.remove(x)	Remove first item from list with value x.	<pre>my_list = [5, 8, 12] my_list.remove(8)</pre>
list.pop()	Remove and return last item in list.	<pre>my_list = [5, 8, 12] my_list.pop()</pre>

<code>list.pop()</code>	Remove and return last item in list.	<code>val = my_list.po</code>
<code>list.pop(i)</code>	Remove and return item at position <code>i</code> in list.	<code>my_list = [5, 8,</code> <code>val = my_list.po</code>

Modifying elements

List method	Description	Code example
<code>list.sort()</code>	Sort the items of list in-place.	<code>my_list = [14, 5, 8</code> <code>my_list.sort()</code>
<code>list.reverse()</code>	Reverse the elements of list in-place.	<code>my_list = [14, 5, 8</code> <code>my_list.reverse()</code>

Miscellaneous

List method	Description	Code example
<code>list.index(x)</code>	Return index of first item in list with value <code>x</code> .	<code>my_list = [5, 8, 14</code> <code>print(my_list.index</code>
<code>list.count(x)</code>	Count the number of times value <code>x</code> is in list.	<code>my_list = [5, 8, 5,</code> <code>print(my_list.count</code>

PARTICIPATION
ACTIVITY

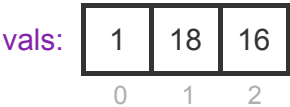
6.4.1: In-place modification using list methods.

Start

☐ 2x speed

```
vals = [1, 4, 16]

vals.append(9)
vals.insert(2, 18)
value = vals.pop()
vals.remove(4)
vals.remove(55) # Error
```



Captions ^

1. vals is a list containing elements 1, 4, and 16.
2. The statement vals.append(9) appends element 9 to the end of the list.
3. The statement vals.insert(2, 18) inserts element 18 into position 2 of the list.
4. The statement vals.pop() removes the last element, 9, from the list.
5. The statement vals.remove(4) removes the first instance of element 4 from the list.
6. The statement vals.remove(55) attempts to remove the first instance of element 55 from the list, which causes the program to produce an error.

Try 6.4.1: Amusement park ride reservation system.

The following (unfinished) program implements a digital line queuing system for an amusement park. The program allows a rider to reserve a place in line without actually having to wait. The rider simply enters a name into a program. If the rider purchases a VIP pass, they get to skip past the common riders up to the last VIP rider in line. VIPs board the ride first. The average wait time for a Disneyland ride is about 45 minutes ([Disneyland Crowd Levels](#)), this might be a good idea. In a manual system, an employee manually selects when the ride is dispatched (thus removing the next rider in line).

Complete the following program, as described above. Once finished, add the following commands to the program:

- The rider can enter a name to find the current position in line. (Hint: Use the list.index() method.)
- The rider can enter a name to remove the rider from the line.



▶ Run

```
1  riders_per_ride = 3  # Num riders per ride to dispatch
2
3  line = []  # The line of riders
4  num_vips = 0  # Track number of VIPs at front of line
5
6  # FIXME: Add menu commands to find and remove riders.
7  menu = "(1) Reserve place in line.\n"  # Add rider to line
8         "(2) Reserve place in VIP line.\n"  # Add VIP
9         "(3) Dispatch riders.\n"  # Dispatch next ride car
10        "(4) Print riders.\n"
11        "(5) Exit.\n\n")
12
13  user_input = input(menu).strip().lower()
14
15  while user_input != "5":
16      if user_input == "1":  # Add rider
17          name = input("Enter name: ").strip().lower()
18          print(name)
19          line.append(name)
20
21      elif user_input == "2":  # Add VIP
22          # FIXME: Add new rider behind last VIP in line
```

CONSOLE

→

**Model Solution** ▼**CHALLENGE
ACTIVITY**

6.4.1: List methods.

763650.1722066.qx3zqy7

Start

Integer `num_raw` is read from input, representing the number of integers to be read next. Read input and append each integer to `raw_list`.

[Click here for example](#) ▼

```
1 num_raw = int(input())
2
3 raw_list = []
4
5 """ Your code goes here """
6
7 print(raw_list)
```

1

2

Check**Next level**

View solution ▼ *(Instructors only)*